

Timely-Dataflow

Bringing expressiveness back to distributed computing

19/05/2021

Matteo Ceccarelli – UNIBZ

Roadmap

- Dataflow models
- Timely dataflow
- Experiments
- Summary

MapReduce and Streaming

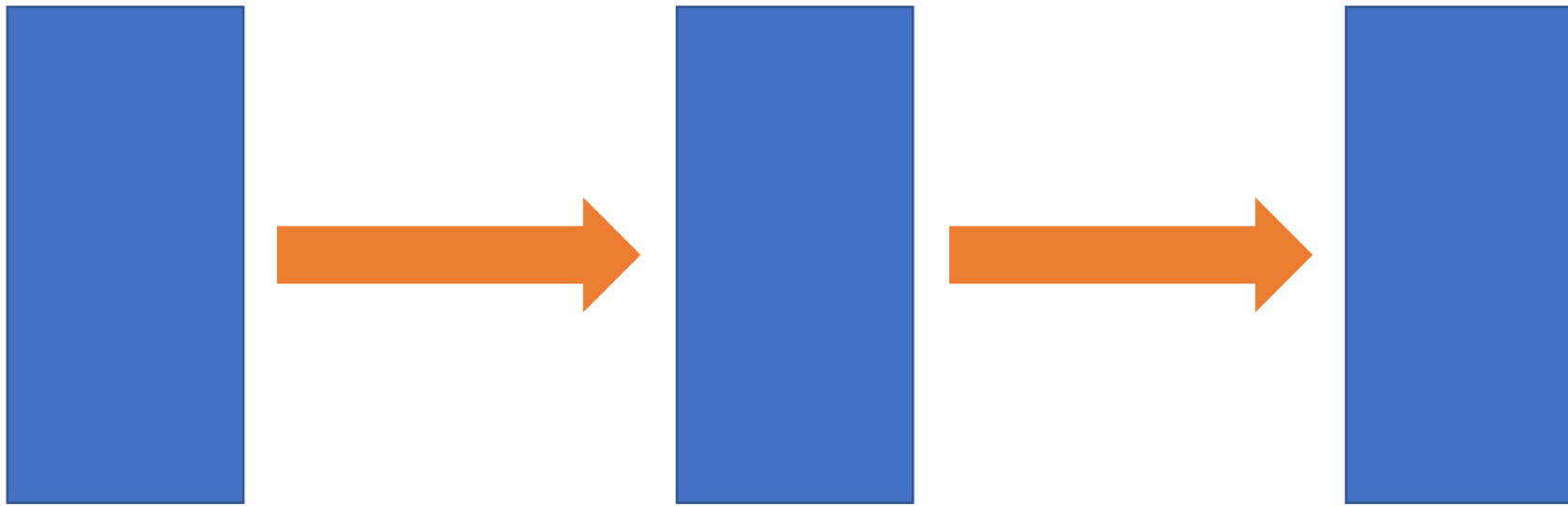
MapReduce

- Several processors
- Each with limited memory
- Minimize the number of rounds

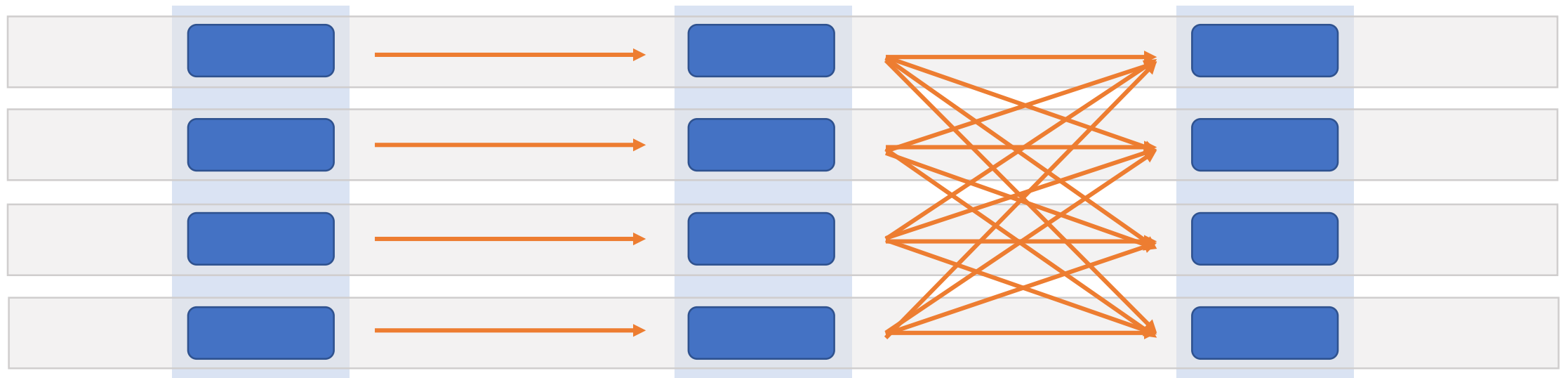
Streaming

- A single processor
- Limited memory
- Minimize the number of passes

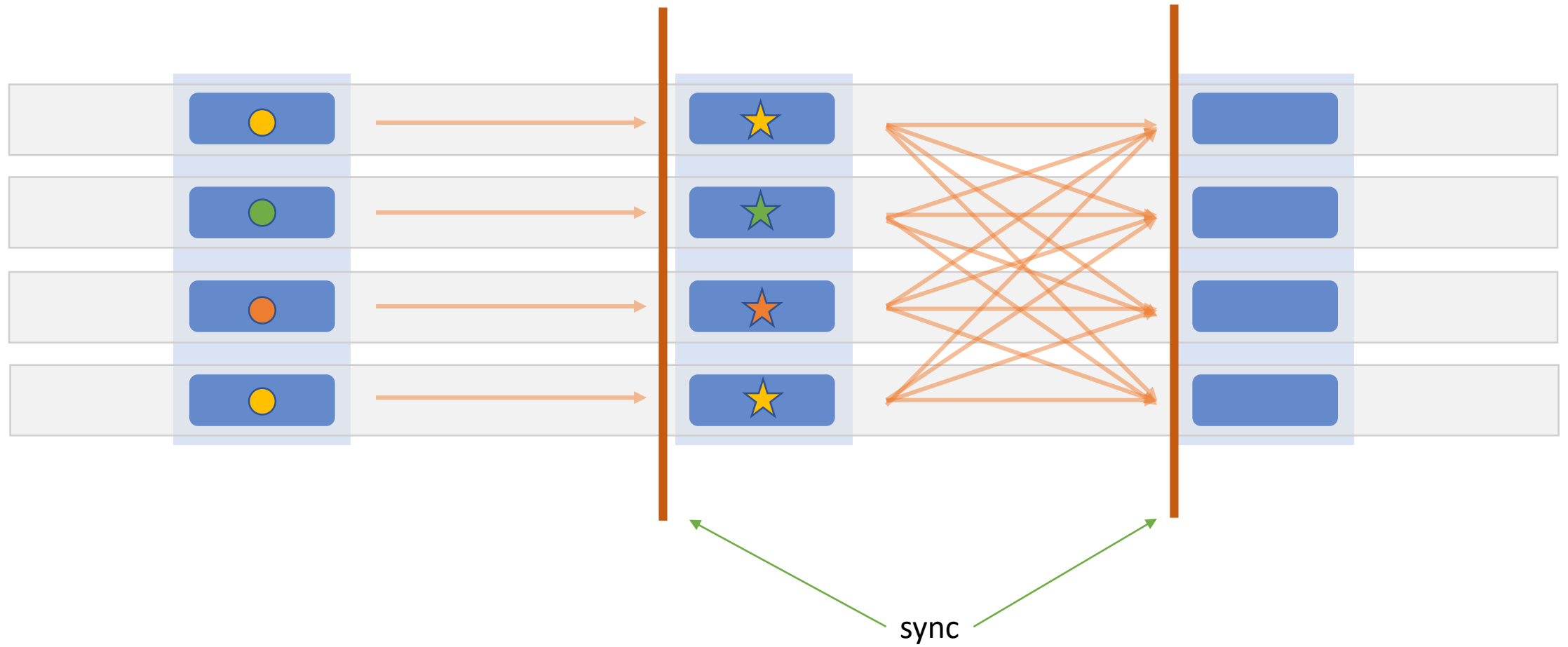
Dataflow models



Dataflow models – splitting across multiple machines



Dataflow models - MapReduce



Dataflow models - Streaming



MapReduce vs. Streaming

MapReduce

- Easy aggregation
- High latency
- Cannot maintain state
- Finite size input

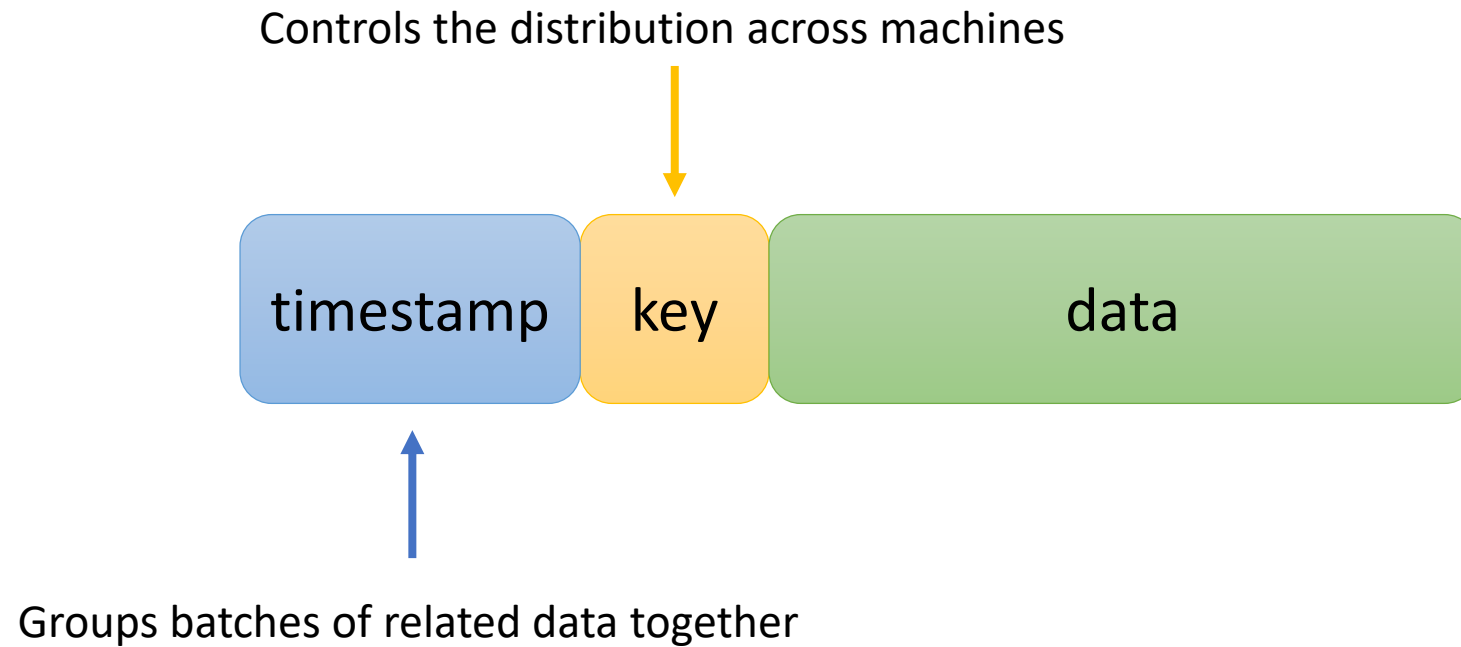
Streaming

- Low latency
- Can maintain state
- Works with unbounded input
- Difficult aggregation
 - *When have we seen all data?*

Bringing together batch and streaming?

- Streaming: *When have we seen all data?*
- We need a mechanism to group batches of messages

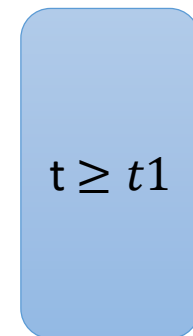
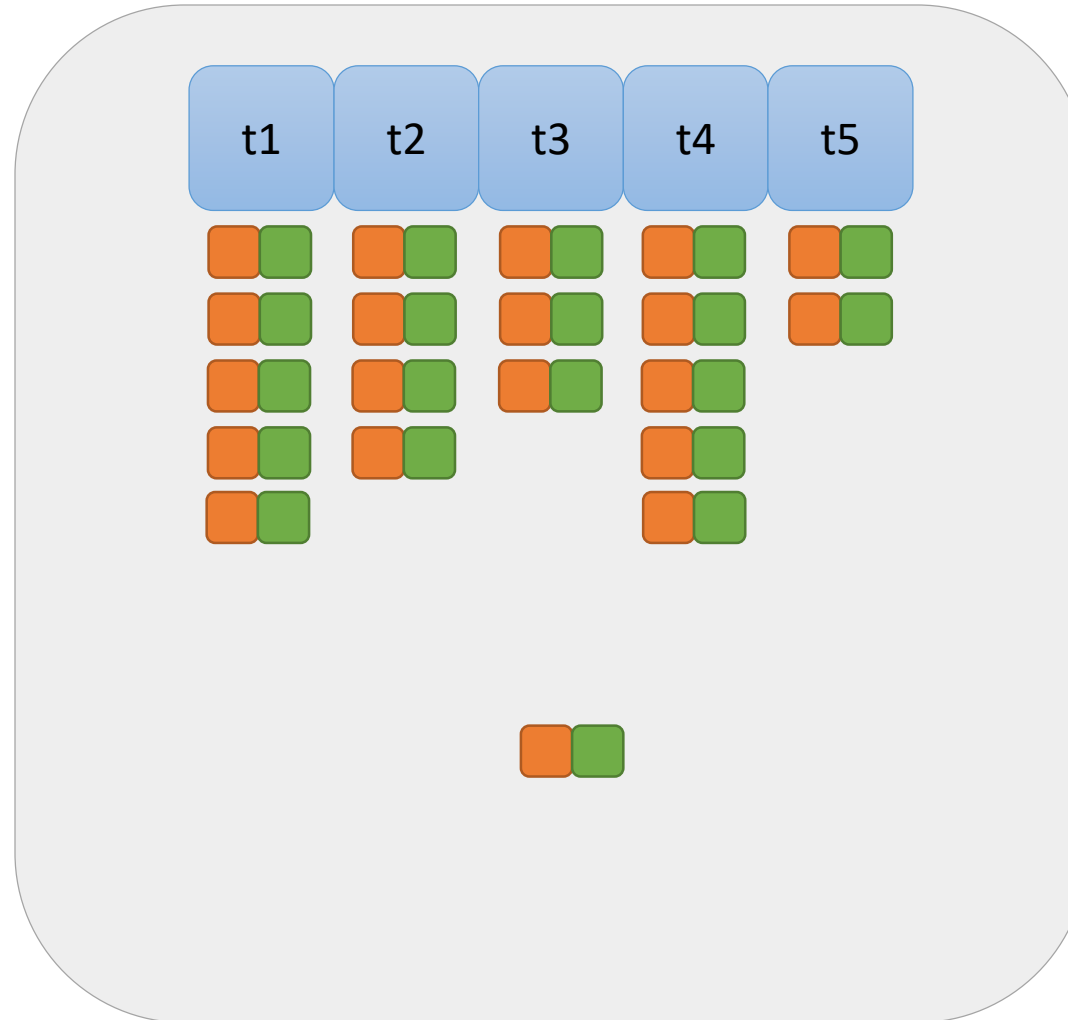
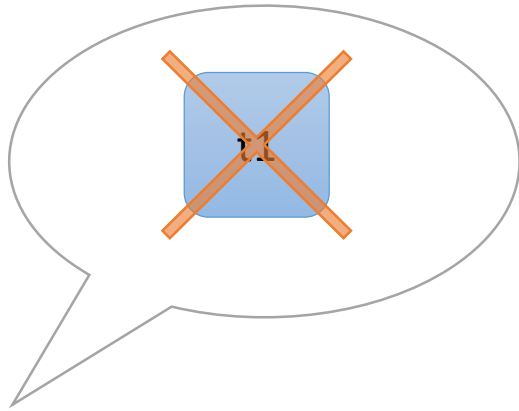
Introducing timestamps



Timestamps

- Timestamps are *ordered*
- The system keeps track of all the timestamps active at any given time
- Inputs can declare that a timestamp will no longer be emitted
- The system notifies all the workers of such an event
- Workers can then act based on this information

Timestamps – accumulating data



Timestamps

- Using timestamps introduces synchronization to streaming
- What's the point?
- This type of synchronization is *opt-in*!
- Communication and computation can overlap

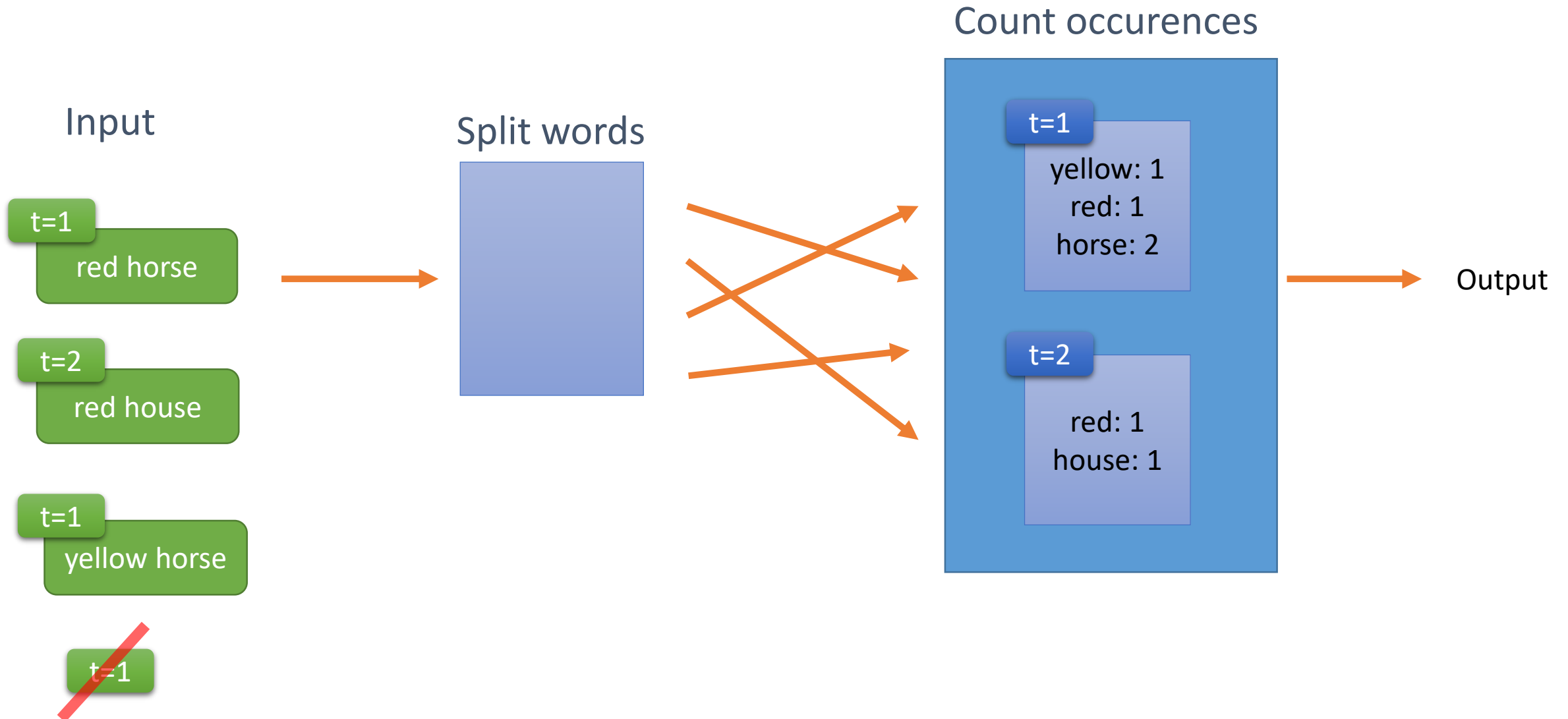
Timely-Dataflow

- Aggregation within timestamps is possible
- Low latency/high throughput possible
- Can maintain state
- Can work with unbounded inputs
- Iterative algorithms

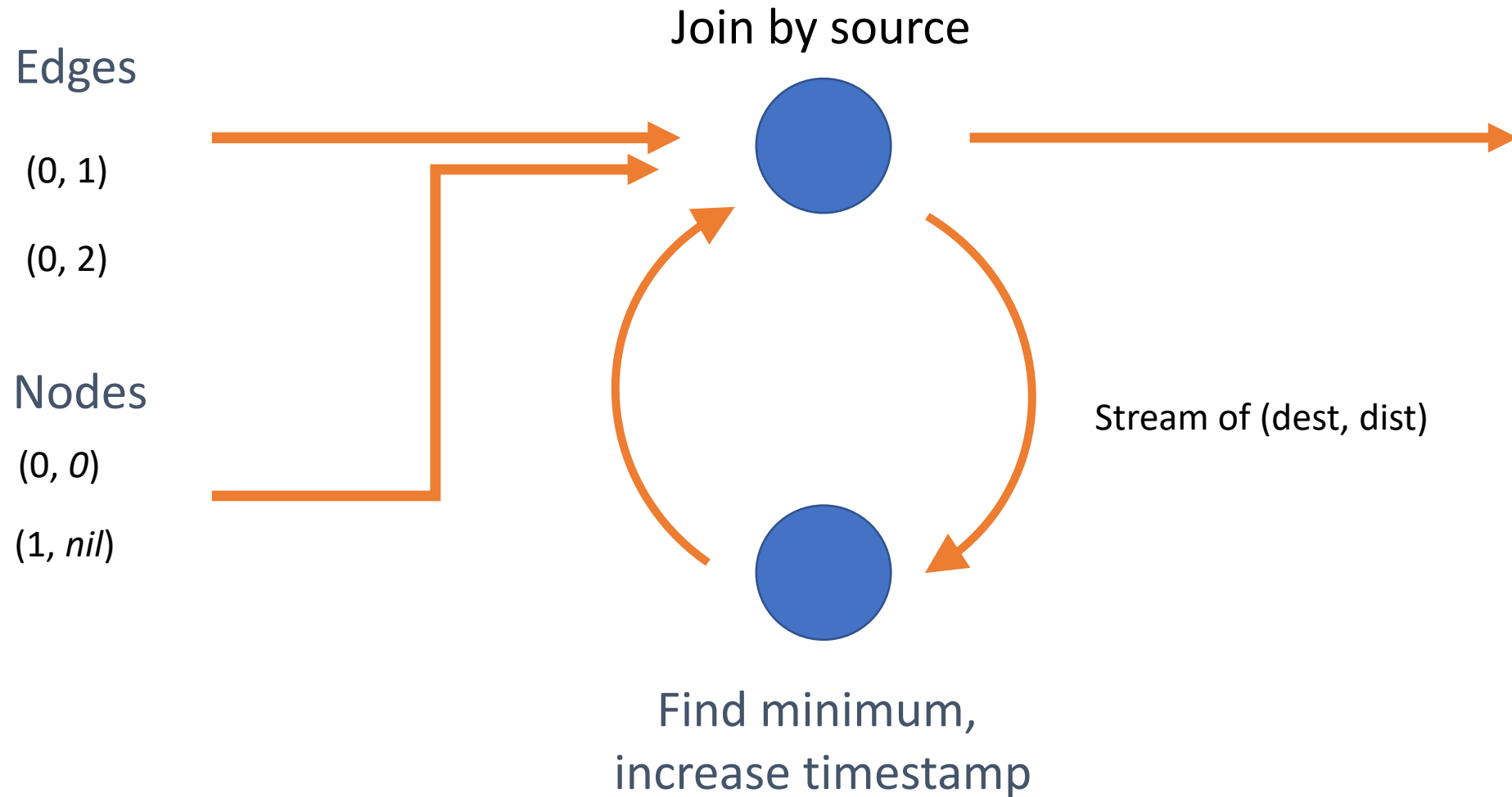
Murray, D. G., McSherry, F., Isaacs, R., Isard, M., Barham, P., & Abadi, M. 2013.
Naiad: a timely dataflow system. In *SoSP* ACM.

McSherry, F., Murray, D. G., Isaacs, R., & Isard, M. 2013.
Differential Dataflow. In *CIDR*.

Simple linear dataflow (word count)



Iterative dataflow (BFS)



Cost model

- The MapReduce model is ill suited: there are no (explicit) rounds
- The Streaming model does not account for parallelism
- Algorithms are analyzed under the Massively Parallel Computation (MPC) model
 - Minimize the *load* (number of messages received by each worker)

How does a Timely-Dataflow program look like?

Word count in a MapReduce like way

```
1 docs
2     .flat_map(|doc| {
3         doc.split_whitespace().map(|w| (w, 1))
4     })
5     .aggregate(
6         |_w, val, agg| { *agg += val; },    // state update
7         |w, agg: u32| (w, agg),            // build result
8         |w| w.hash()                       // shuffling key
9     )
10    .inspect(|(w, c)| println!("Word: {} Count: {}", w, c));
```

How does a Timely-Dataflow program look like?

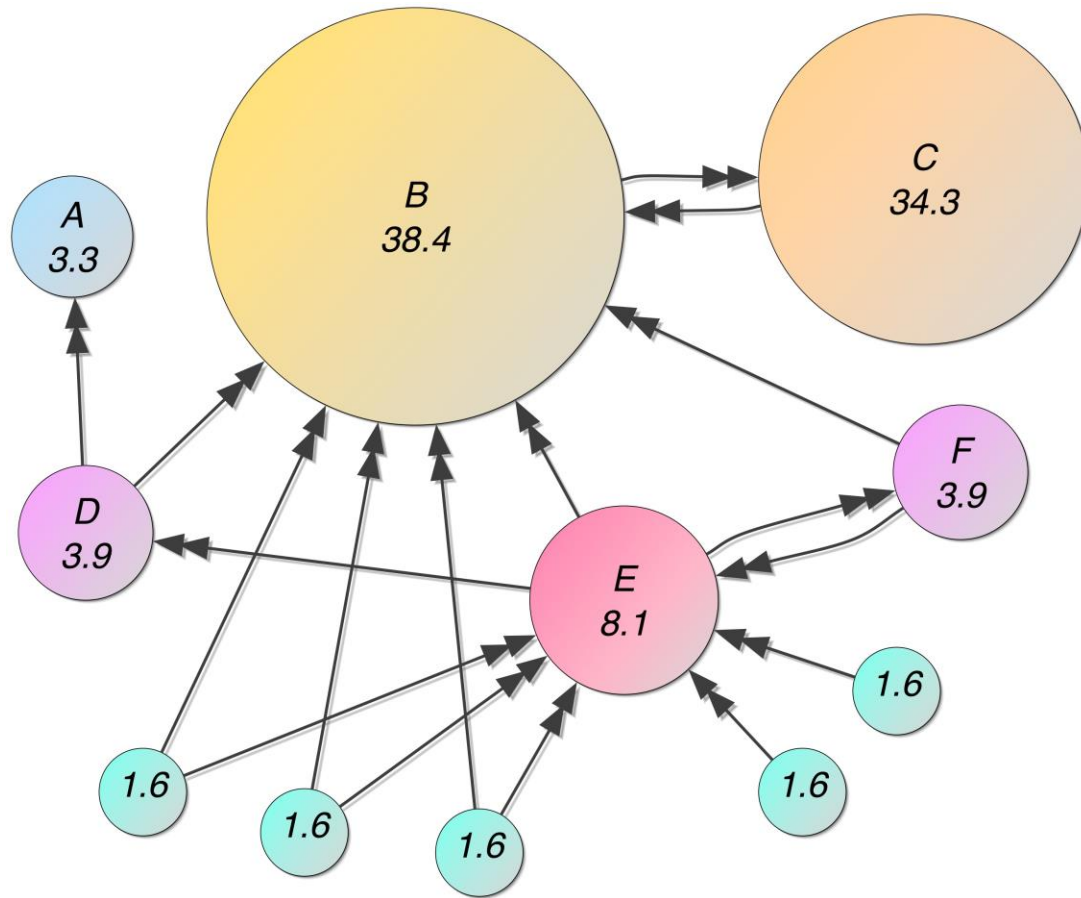
- Things can get arbitrarily complex
- Problems you are dealing with may be complex
- Streams with timestamps give you the flexibility of dealing with complex situations



Expressive framework!

```
candidates
    .approximate_distinct_atomic(
        ExchangePact::new(move |pair: &(K, K)| {
            let row = pair.0.route() % u64::from(matrix.rows);
            let col = pair.1.route() % u64::from(matrix.columns);
            matrix.row_major(row as u8, col as u8)
        }),
        Arc::clone(&bloom_filter),
    )
    .unary(PipelinePact, "count-matching", move |_, _| {
        let mut pl =
            ProgressLogger::new(Duration::from_secs(60), "comparisons".to_owned(), None);
        move |input, output| {
            input.for_each(|t, d| {
                let _pg = ProfileGuard::new(
                    logger.clone(),
                    t.time().to_step_id(),
                    1,
                    "distance_computation",
                );
                let mut data = d.replace(Vec::new());
                let count = data
                    .drain(..)
                    .filter(|(lk, rk)| {
                        let lv = &global_left[lk];
                        let rv = &global_right[rk];
                        sim_pred(lv, rv)
                    })
                    .count() as u64;
                pl.add(count);
                let mut session = output.session(&t);
                session.give(count);
            });
        }
    })
    .stream_sum()
```

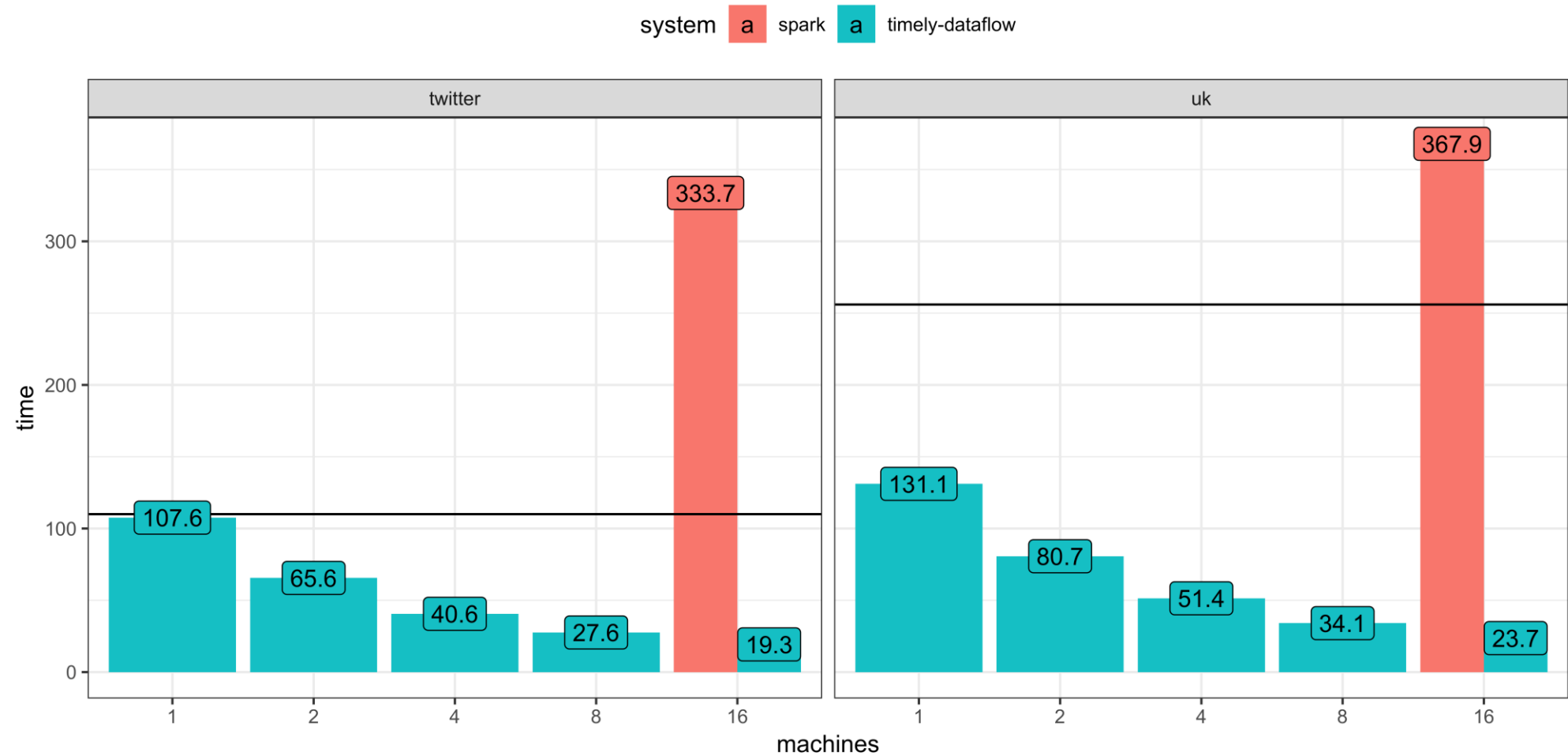
PageRank



- Assigns a numerical value to vertices
- Proceeds iteratively, propagating information along the edges

Experiments

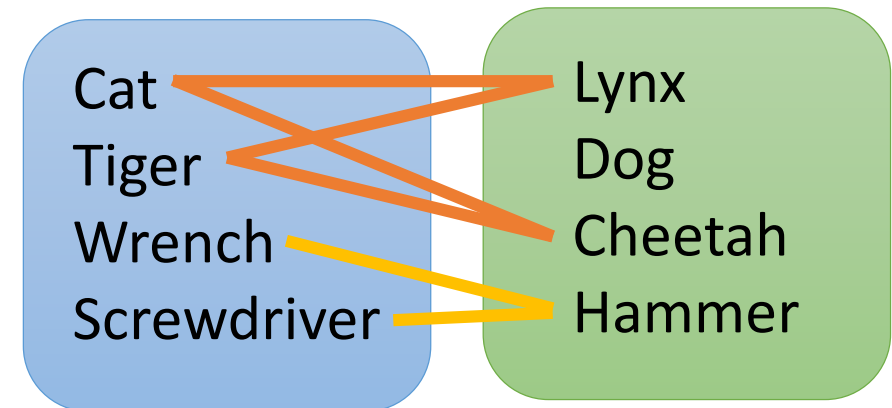
20 iteration of Pagerank - Twitter: 1.4B edges - UK: 3.7B edges



Each machine has 8 cores

Similarity joins

- Two sets
- A measure of similarity
- Find all pairs above a given similarity
- The output may be of size quadratic wrt the input!
- Potential problems with communication



DANNY: *Distributed Approximate Near Neighbours, Yo!*

- Use timestamps to break down work
- Keep communication linear at any given time

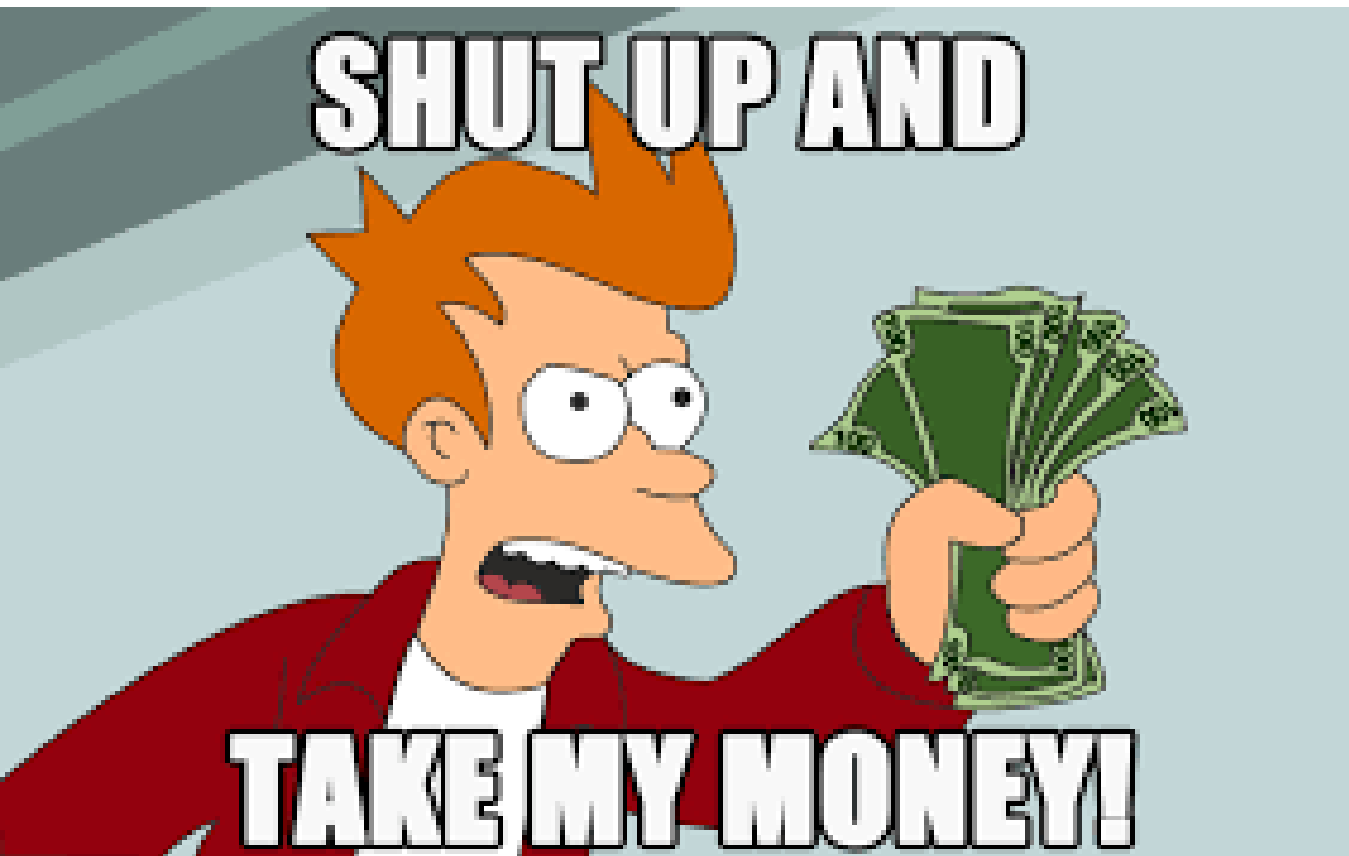
Spark



Timely-Dataflow

≈10 minutes

On a 3 million elements join, using 5 machines with 8 cores each



Why don't we just
use timely-dataflow
for all things?



PROS

CONS

Timely Dataflow

- Expressive model
- High throughput/low latency
- Implemented in Rust
- Manage your own memory

- Sometimes difficult to work in
- No fault tolerance
- Implemented in Rust
- Manage your own memory

Spark

- Simple programming model
- Widely used
- Fault tolerance
- Can be used in Scala/Java/Python/R
- Rich machine learning library

- In some cases the performance is limited
- Iterative computations are cumbersome to express
- Implemented on top of the JVM: good performance but obscure memory management

mceccarello@unibz.it

Suggested readings:

- <https://github.com/frankmcsherry/blog/>
- <http://www.frankmcsherry.org/assets/COST.pdf>
- <https://materialize.com/>